

MISIÓN

LLEVAR EL ERP A UN ENTORNO CONFIABLE

2 frontends Next.js en Docker: **ERP :3000** y **ANUNCIOS :3001**

Backend **FastAPI/Gateway** + 3 microservicios

PostgreSQL local reemplazado por **Supabase** gestionado

CI ya hace **lint + build**



NIVEL 1

PRUEBAS FUNCIONALES

Pruebas unitarias con **Vitest + React Testing Library**
formularios, validaciones y duplicados

Pruebas de integración con **Vitest**
flujos dentro de cada Next.js

Pruebas E2E con **Playwright** login, alta postulante,
publicación y detalle de anuncio

Pruebas de **backend** cuando exista código real
pytest + httpx y pytest



NIVEL 2

PRUEBAS NO FUNCIONALES

Rendimiento del frontend con **Lighthouse CI**

Latencia y throughput del Gateway con **k6**

Seguridad y privacidad: **TLS, JWT/CORS**,
protección de PII y auditorías **npm audit/pip-audit**

Accesibilidad y calidad con **axe-core + Playwright**
multi-browser y resiliencia con usuarios reales



ESTRATEGIA DE DESPLIEGUE

EVOLUCIÓN CONTROLADA DESDE DESARROLLO HASTA UN ENTORNO DEMOSTRABLE



CUANDO EXISTA BACKEND: SOLO GATEWAY EXPUESTO;
MICROSERVICIOS Y BD SIN TRÁFICO DIRECTO DE INTERNET.



POWER-UPS

STACK OPERATIVO Y CONTROLES

PARA DESPLIEGUES REPETIBLES

Automatización con [GitHub Actions](#)
+ [Docker/Compose](#) + [Supabase](#)

Realizar Migraciones

CI antes de CD, [/health](#)
y menor exposición

Imágenes por commit/tag para
rollback y auditoría de dependencias
con [npm audit](#) y [pip-audit](#)



ruta final

laC en AWS: infraestructura reproducible

Terraform como laC:
plan revisable antes de aplicar

ECR para imágenes;
App Runner o ECS Fargate para contenedores

ALB termina TLS y
Gateway mantiene auth/CORS

RDS uso de BD;
Secrets Manager guarda conexión/JWT

Alcance: demostrar laC reproducible
VPC compleja ni multi-cuenta

